

Qué son

Los 4 pilares son 4 principios que todo código orientado a objetos debe de tener.

Se consideran cultura general, así que es esencial aprenderlos y entenderlos a la perfección.

Suelen ser preguntas de entrevista.

Herencia

Los humanos heredamos gran parte de nuestro comportamiento y físico de nuestros padres por genética.

Al igual que los humanos, las clases pueden *heredar* características de otras clases.

Esto incluye sus `atributos` y `métodos`.

A la clase que hereda cosas de otra le llamamos `clase hija`, mientras que a la clase de la que hereda le llamamos `clase padre`.

Por ejemplo, creemos una clase `Vehicle` que tenga 2 métodos: `turn_on` y `turn_off`:

```

1 class Vehicle:
2     is_on = False
3     wheel_number = 0
4
5     def turn_on(self):
6         self.is_on = True
7         print(f"Vehicle with {self.wheel_number} wheels is on")
8
9     def turn_off(self):
10        self.is_on = False
11        print(f"Vehicle with {self.wheel_number} wheels is off")

```

Copiar

Ahora creamos 2 clases que hereden de `Vehicle`:

```

1 class Car(Vehicle):
2     wheel_number = 4
3
4
5 my_car = Car()
6 my_car.turn_on()
7 my_car.turn_off()

```

Copiar

```

1 class Bike(Vehicle):
2     wheel_number = 2
3
4
5 my_bike = Bike()
6 my_bike.turn_on()
7 my_bike.turn_off()

```

Copiar



Vehicle with 4 wheels is on
Vehicle with 4 wheels is off



Vehicle with 2 wheels is on
Vehicle with 2 wheels is off

Como podemos ver, las instancias de las clases `Car` y `Bike` tienen los mismos *métodos* y *atributos* que la clase `Vehicle` debido a que `Car` y `Bike` heredan de ella.

Son sus `clases hijas`, y `Vehicle` es la `clase padre`.

Sin embargo, también tienen valores de `wheel_number` distintos porque sobre-escribieron el de `Vehicle`.

Herencia Multiple

Python es de los pocos lenguajes en donde una clase puede `heredar` de **n cantidad de clases** y no solo de 1.

Por ejemplo, puedo tener tres clases padre:

```
1 class WalkerMixin:
2     def walk(self):
3         print("I'm walking!")
4
5 class RunnerMixin:
6     def run(self):
7         print("I'm running!")
8
9 class FlyerMixin:
10    def fly(self):
11        print("I'm flying!")
```

Copiar

Y una clase que herede de todas a la vez:

```
1 class Superman(WalkerMixin, RunnerMixin, FlyerMixin):
2     pass
3
4 clark_kent = Superman()
5 clark_kent.walk()
6 clark_kent.run()
7 clark_kent.fly()
```

Copiar



I'm walking!

I'm running!

I'm flying!

En caso de que dos (o mas) de estas clases tengan métodos o atributos con el mismo identificador, la primera tendrá prioridad y sobre-escribirá a las de las posteriores.

Por ejemplo:

```

1 class ClassA:
2     name = "A"
3
4     def my_method():
5         print("Hello")
6
7 class ClassB:
8     name = "B"
9
10    def my_method():
11        print("Goodbye")

```

Copiar

Ahora veamos cual name es el que queda según el orden de herencia:

```

1 class ClassC(ClassA, ClassB):
2     def print_name(self):
3         print(f"My name is {self.name}")
4
5 my_c = ClassC()
6 my_c.print_name()
7 my_c.my_method()

```



My name is A
Hello

```

1 class ClassC(ClassB, ClassA):
2     def print_name(self):
3         print(f"My name is {self.name}")
4
5 my_c = ClassC()
6 my_c.print_name()
7 my_c.my_method()

```



My name is B
Goodbye

Clases Abstractas

Otra manera de lograr esto es usando `clases abstractas`.

Estas son clases que no se pueden instanciar, sino que se usan *exclusivamente* para ser clases padre en una jerarquía de herencia.

Así mismo, pueden tener `métodos abstractos` que **deben ser sobre-escritos** por sus clases hijas.

Son como *contratos*: si heredas de mí, tienes que implementar este funcionamiento.

Por ejemplo, creemos una clase abstracta de `Animal` con un método abstracto de `reproduce`.

```
1  from abc import ABC, abstractmethod
2
3
4  class Animal(ABC):
5      def breath(self):
6          pass
7
8      def born(self):
9          pass
10
11     @abstractmethod
12     def reproduce(self):
13         # Todas las especies de animales deben reproducirse
14         # Pero lo pueden hacer de distintas maneras
15         pass
16
17     class AsexualAnimal(Animal):
18         def reproduce(self):
19             print("Reproducing in an asexual manner")
20
21     class SexualAnimal(Animal):
22         def reproduce(self, mate):
23             print(f"Reproducing in a sexual manner with {mate}")
24
25     class OtherAnimal(Animal):
26         pass
27
28
29     asexual_animal = AsexualAnimal()
30     asexual_animal.reproduce() # -> Reproducing in an asexual manner
31
32     sexual_animal_a = SexualAnimal()
33     sexual_animal_b = SexualAnimal()
```

Copiar

```
34 sexual_animal.reproduce(sexual_animal_b) # -> Reproducir
35
36 animal = Animal() # va a fallar porque Animal es una clase
37 other_animal = OtherAnimal() # va a fallar porque no se puede
```

```
1 from abc import ABC, abstractmethod
2
3 class ReportEmailer(ABC):
4     @abstractmethod
5     def generate_report():
6         # genera el reporte
7         pass
8
9     def email_report():
10        # envia el report por email
11        pass
12
13    def generate_report_and_email(self):
14        self.generate_report()
15        self.email_report()
```

Copiar

Encapsulamiento

En la mayoría de lenguajes de programación existe un nivel de acceso o de protección para los métodos y atributos.

`public` : pueden ser accesados desde cualquier lugar.

`protected` : pueden ser accesados desde los métodos de la clase y sus clases hijas.

`private` : pueden ser accesados solo desde los métodos de la clase.

Esto se hace para proteger al código de un uso inadecuado por parte de los desarrolladores.

```
1 class BankAccount():
2     balance = 0
3
4     def __subtract_balance(self, amount):
5         self.balance -= amount
6
7     def __add_balance(self, amount):
8         self.balance += amount
9
10    def send_money_to_account(self, account, amount):
11        self.__subtract_balance(amount)
12        account.__add_balance(amount)
13
14
15 bank_account = BankAccount()
16 bank_account.__add_balance(5000)
```

Python carece de esta funcionalidad, pero si *hipotéticamente*, funcionaría así:

```
1 class Person:
2     name: string #public
3     _date_of_birth: datetime #protected
4     __sex: string # private
5
6     def __init__(self, name, date_of_birth, sex)
7         self.name = name
8         self._date_of_birth = date_of_birth
9         self.__sex = sex
10
11 class Worker(Person):
12     def print_date_of_birth(self):
13         print(self._date_of_birth)
14
15     def print_sex(self):
16         print(self.__sex)
17
18 my_person = Person("Juan", "2003/02/02", "Male")
19 print(my_person.name) # -> Juan
20 print(my_person._date_of_birth) # -> error, ya que
21 print(my_person.__sex) # -> error, ya que __sex es
22
23 my_worker = Worker("Joan", "1984/05/06", "Female")
24 my_worker.print_date_of_birth() # -> 1984/05/06
25 my_worker.print_sex() # -> error, ya que __sex es
```

Copiar

Debido a que Python carece de esta protección, (y el ejemplo de arriba es hipotético - en realidad todos los casos funcionarían), lo que se usa *como estándar* es usar guiones bajos en sus identificadores para especificar su nivel de protección.

`name` es publico.

`_name` es protected.

`__name` es privado.

`get_age` es publico.

`_get_age` es protected.

`__get_age` es privado.

Abstracción

La abstracción permite poder trabajar con objetos de manera efectiva sin necesidad de conocer todos los detalles internos de los mismos.

Por ejemplo, cuando manejamos un carro, no necesitamos saber cómo funciona su motor a nivel interno.

Solo necesitamos saber cómo usar los pedales y la manivela.

Para ponerlo en palabras simples, es el acto de hacer que los métodos "*públicos*" de nuestros objetos sean lo más claros y sencillos posibles, y dividir su lógica entre métodos "*privados*". (uso comillas porque, reitero, en Python no existe tal cosa).

```
1 class Person:
2     name: string
3     _date_of_birth: datetime
4
5     def __init__(self, name, date_of_birth)
6         self.name = name
7         self._date_of_birth = date_of_birth
8
9     def get_age(self)
10         return today() - self._date_of_birth`
```

Copiar

Polimorfismo

Viene de poli (varias) morfo (formas).

Se refiere a que una sintaxis o método con el mismo `identificador` pueden tener comportamientos o formas distintas dependiendo del contexto en que se usen.

Por ejemplo, la función `range` dependiendo del numero de parámetros que tenga.

Otro ejemplo es el símbolo de `+`.

En el caso de los objetos, podemos estandarizar ciertos identificadores para poder usarlos de manera intercambiable con clases que no estén relacionadas.

Por ejemplo, el método `reproduce` del ejemplo de arriba es un caso de polimorfismo.

Aquí otro ejemplo, podemos tener una lista de objetos de tipo Vehículo y de tipo Computadora, y que todos tengan un método de "encender" y "apagar":

```
1 class Vehiculo:
2     is_on: bool
3     ruedas: int
4
5     def encender(self):
6         self.is_on = True
7         print("Vehiculo encendida")
8
9     def apagar(self):
10        self.is_on = False
11
12 class Computadora:
13     is_on: bool
14
15     def encender(self):
16         self.operative_system = "Windows"
17         self.is_on = True
18         print("Computadora encendida")
19
20     def apagar(self):
21         self.is_on = False
```

Copiar

```
1 object_list = [
2     Vehiculo(),
3     Vehiculo(),
4     Computadora(),
5     Computadora(),
6 ]
7
8 for object in object_list:
9     object.encender()
```

Copiar